ЧАСТЬ 2 ИЗ 3

RUST

Данные, коллекции и функции

С какими данными работает программа — и кто за них отвечает

Числа и строки	struct и enum	Коллекции	Итераторы
Замыкания	Generics	Ownership	Borrowing

Числа и строки подробнее

```
let a: u8 = 255;              // 0..=255
let b: i64 = -9_000_000_000;
let idx: usize = 3;           // индексы, длины
let f: f32 = 1.5;

let next = a.checked_add(1);  // None

let s: String = String::from("привет");
let r: &str = &s;             // срез
println!("{}", r.len());            // 12 байт
println!("{}", s.chars().count());  // 6 символов
```

u8 = 255, затем + 1: debug-сборка — паника, release — «провернётся» в 0

`i8 … i128, u8 … u128`

Размер — в имени типа, u — без знака

`usize`

Размер указателя: так задаются индексы и длины

`checked_add → Option`

Переполнение обрабатываем явно

`String / &str`

Владеющий текст в куче / ссылка на текст. len() — байты UTF-8

Свои типы: struct и enum

```rust
use std::f64::consts::PI;

struct Point { x: i32, y: i32 }

enum Shape {
    Circle(f64),              // вариант с данными
    Rect { w: f64, h: f64 },  // именованные поля
}

fn area(s: Shape) -> f64 {
    match s {
        Shape::Circle(r) => PI * r * r,
        Shape::Rect { w, h } => w * h,
    }
}
```

struct — «и»

Точка — это x и y одновременно

enum — «или»

Фигура — круг или прямоугольник; вариант хранит свои данные

match по enum

Забыли вариант — программа не компилируется

Option<T> и Result<T, E> из части 3 — тоже enum

код: part2/03_struct_enum.rs

Коллекции: Vec, HashMap, HashSet

`Vec<T>`

динамический массив

```
let mut v = vec![1, 2, 3];
v.push(4);
v.remove(0);    // [2, 3, 4]
let a = v[0];   // 2
v.get(10);      // None
```

v[10] — паника, v.get(10) — None

`HashMap<K, V>`

значение по ключу

```
let mut ages = HashMap::new();
ages.insert("Анна", 21);
ages.insert("Олег", 23);
ages.get("Анна"); // Some(&21)
ages.remove("Олег");
```

get → Option: ключа может не быть. Порядок не гарантирован

`HashSet<T>`

множество

```
let mut tags = HashSet::new();
tags.insert("rust");
tags.insert("rust"); // дубль
tags.contains("rust"); // true
tags.len();            // 1
```

Хранит только уникальные значения

```
use std::collections::{HashMap, HashSet};
```

Массив [T; N] из части 1 — фиксированная длина. Коллекции растут во время работы.

Итераторы: три способа и next()

```
let mut v = vec![10, 20, 30];

for x in v.iter() { println!("{x}"); } // &i32
for x in v.iter_mut() { *x += 1; }     // &mut i32

let mut it = v.iter();
it.next();   // Some(&11)
it.next();   // Some(&21)

for x in v.into_iter() { println!("{x}"); } // i32
// дальше v недоступен — вектор потреблён
```

for

Вызывает next(), пока не вернётся None

iter() → &T

Читаем, коллекция на месте

iter_mut() → &mut T

Меняем элементы; вектор объявлен mut

into_iter() → T

Забирает элементы, коллекция потреблена

Тройка «прочитать — изменить — забрать» ещё встретится в ownership.

Замыкания и цепочки итераторов

```
let limit = 10;                  // внешняя переменная
let add_one = |x: i32| x + 1;    // замыкание — значение
add_one(4);                      // 5

let small: Vec<i32> = vec![1, 5, 12]
    .iter()
    .copied()                    // &i32 → i32
    .filter(|x| *x < limit)      // захват limit
    .map(add_one)
    .collect();                  // запуск цепочки
// small == [2, 6]
```

|x| x + 1

Параметры между чертами, после них — тело

Захват окружения

По &, по &mut или с владением; move — забрать явно

Ленивость

filter и map ничего не делают до collect()

Функциональный стиль в Rust

Generics: декларация, инстанцирование, использование

1 · Декларация

```
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut max = &list[0]; // срез не пустой
    for item in list {
        if item > max { max = item; }
    }
    max
}

struct Point<T> { x: T, y: T }
```

2 · Инстанцирование и 3 · использование

```
let a = largest(&[1, 7, 3]);
// T = i32 — выведен компилятором

let b = largest::<f64>(&[0.5, 2.5]);
// T = f64 — указан явно

let p = Point { x: 1.0, y: 2.0 };
// Point<f64>
```

T — заполнитель типа. Компилятор создаёт версию под каждый тип: во время выполнения платы нет.

Trait bounds: ограничиваем T

```rust
use std::fmt::Display;

// fn print_bad<T>(v: T) { println!("{v}"); }
// E0277: `T` doesn't implement `Display`

fn show<T: Display>(v: T) {
    println!("[{v}]");
}

fn show_max<T>(a: T, b: T)
where
    T: Display + PartialOrd,
{
    if a > b { show(a) } else { show(b) }
}
```

T без ограничений

Компилятор не знает, что можно делать с T

T: Display

Любой T, который умеет печататься через {}

+ и where

Несколько ограничений; длинный список — в where

```rust
show(42);          // [42]
show("hello");     // [hello]
show_max(3, 8);    // [8]
```

Что происходит со String в функции?

```
fn takes(s: String) { println!("{s}"); }
fn reads(s: &str)   { println!("{s}"); }

let s = String::from("hello");
reads(&s);          // ок: только читаем
takes(s);           // владение уходит в функцию
println!("{s}");    // ← ошибка
```

```
error[E0382]: borrow of moved value: `s`
```

В других языках объект просто передаётся в функцию.

В Rust параметр String забирает строку себе.

Почему?

Ответ — ownership и borrowing. Это оставшиеся слайды.

Ownership: у значения один владелец

```
{
    let s = String::from("hello"); // владелец s
    println!("{s}");
}   // drop(s): буфер в куче освобождён

fn make() -> String {
    String::from("new")   // владение уходит наружу
}
let t = make();           // теперь владелец — t
```

1. У каждого значения есть владелец: переменная, поле структуры или коллекция.
2. Владелец в каждый момент один.
3. Конец области видимости владельца → drop, ресурсы освобождаются.
4. Владение можно передать — это move (следующий слайд).

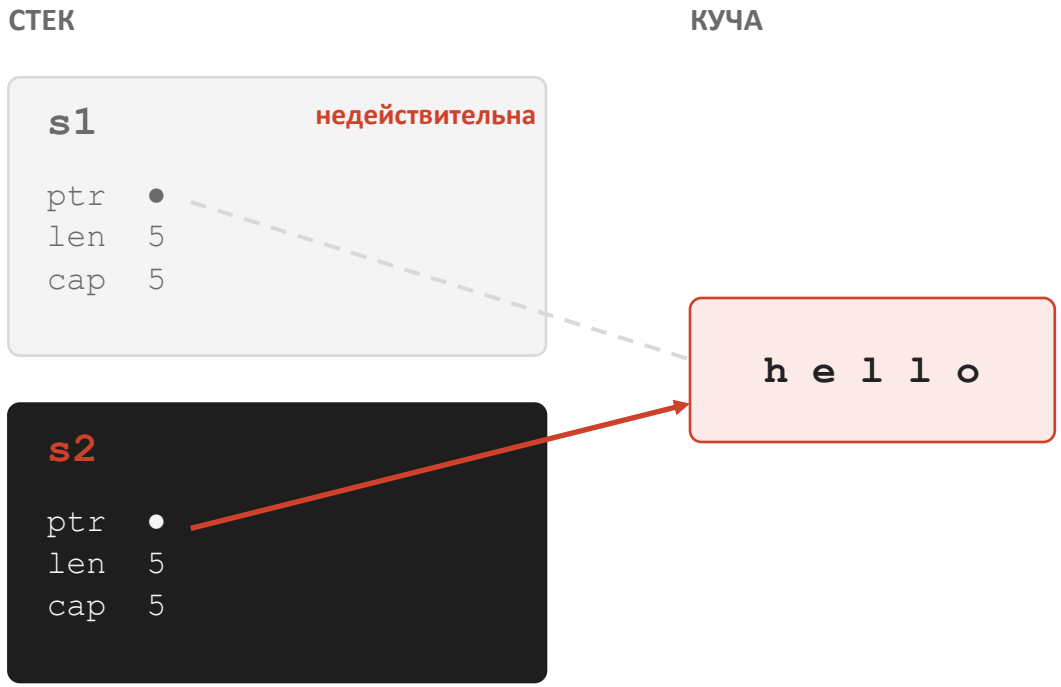
Нет двойного освобождения и обращения к уничтоженным данным — и нет сборщика мусора.

Move: владение переходит

```
let s1 = String::from("hello");
let s2 = s1;          // move
println!("{s2}");     // ок
println!("{s1}");     // ← ошибка
```

```
error[E0382]: borrow of moved value: `s1`
```

Передача аргумента в функцию — тоже move.



Копируются только ptr, len, cap. Текст в куче остаётся на месте — меняется владелец.

Copy и Clone

```
let x: i32 = 10;
let y = x;              // Copy: копия
println!("{x} {y}");    // обе доступны

let s1 = String::from("hi");
let s2 = s1.clone();    // Clone: новый буфер
println!("{s1} {s2}");

let r1 = &s1;
let r2 = r1;            // &T тоже Copy
```

Copy — неявно

Побитовая копия. Это свойство типа, а не стека: числа, bool, char, &T, кортежи и массивы из Copy-типов. Тип с Drop не может быть Copy.

Clone — явно

Вызов .clone() виден в коде. Для String — новый буфер в куче, то есть реальная цена.

Правило: передавать по ссылке, clone() — осознанно.

код: part2/12_copy_clone.rs

Borrowing: &T и &mut T

```
fn len_of(s: &str) -> usize {  // &T: читаем
    s.len()
}
fn append(s: &mut String) {    // &mut T: меняем
    s.push_str("!");
}

let mut t = String::from("hi");
let n = len_of(&t);    // владелец — по-прежнему t
append(&mut t);        // нужен let mut
println!("{t} {n}");   // hi! 2
```

&T — чтение

Сколько угодно ссылок, владелец не меняется

&mut T — изменение

Переменная mut, при вызове &mut — видно в коде

&str, а не &String

Принимает и String, и строковый литерал

Заимствование временное: функция вернулась — владелец снова распоряжается данными сам.

Правила borrowing

Много &T — можно

```
let s = String::from("data");
let r1 = &s;
let r2 = &s;
println!("{r1} {r2}");
```

&mut, пока нужна &T, — нельзя

```
let mut s = String::from("data");
let r = &s;
let m = &mut s;    // ← ошибка
m.push('!');
println!("{r}");   // r ещё нужна
```

```
error[E0502]: cannot borrow `s` as mutable
```

Заимствование кончается на последнем использовании ссылки

```
let r = &s;
println!("{r}");   // последнее использование r
let m = &mut s;    // теперь можно
m.push('!');
```

```
компилируется
```

Конфликт есть, только если использование &T и &mut T пересекается во времени.

Ownership ≈ RAII

C++ · RAII

Ресурс привязан к объекту; деструктор освобождает его при уничтожении.

Rust · Ownership + Drop

владелец → конец области видимости → drop() → ресурс освобождён

```
struct Resource { name: String }

impl Drop for Resource {
    fn drop(&mut self) {
        println!("освобождаю {}", self.name);
    }
}

let _a = Resource { name: String::from("A") };
{
    let _b = Resource { name: String::from("B") };
}   // ← освобождаю B
    // конец main ← освобождаю A
```

Вывод программы

```
освобождаю B
освобождаю A
```

Так же закрываются файлы, сокеты и блокировки.

Не у каждого типа есть Drop: i32 — Copy-тип, освобождать нечего.

код: part2/15_drop_raii.rs

ИТОГ ЧАСТИ 2

«В Rust время жизни ресурса привязано к времени жизни владеющего им значения»

Ownership — кто отвечает за ресурс. Borrowing — кто и как пользуется им временно.
Компилятор проверяет оба — без сборщика мусора и без затрат во время выполнения.

В ЧАСТИ 3

`Lifetimes` `Option / Result` `Traits` `Модули и Cargo` `I/O` `Потоки` `Макросы`